Universidade do Minho
Licenciatura em Engenharia Informática

RELATÓRIO

PROGRAMAÇÃO ORIENTADA AOS OBJETOS

DOMUSCONTROL

Grupo 99

Trabalho realizado por:





Ana Patrícia da Silva Machado
(*a111661*)

Carolina Francisca Vieira Dias
(*a111179*)

Maio, 2026

RESUMO

O *DomusControl* é uma aplicação de automação doméstica desenvolvida em Java, no âmbito da unidade curricular de Programação Orientada aos Objetos. Permite gerir casas inteligentes através da criação de divisões, associação de dispositivos de diferentes tipos e definição de automações, escalonamentos e cenários que coordenam ações sobre múltiplos dispositivos. O sistema suporta vários utilizadores com diferentes níveis de permissão, recolhe estatísticas de utilização, persiste o seu estado em ficheiro binário e inclui um motor que sugere automaticamente novas automações e escalonamentos com base no histórico de interações. A arquitetura segue o padrão Model-View-Controller, com a interação feita através de menus em modo texto.

Área de Aplicação: Desenvolvimento no âmbito da Programação Orientada aos Objetos.

Palavras-Chave: Automação doméstica, gestão de dispositivos, deteção de padrões, Java, MVC, Programação Orientada aos Objetos.

ÍNDICE

RESUMO **1**
ÍNDICE **2**
1. INTRODUÇÃO **4**
2. Diagrama de Classes **5**
2.1. Entidades da Aplicação 5
2.1.1. Hierarquia de Dispositivos 5
Figura 1 - Hierarquia de classes da família Device 6
2.1.2. Casas e Divisões 7
Figura 2 - Relação entre House, Room e Device 7
2.1.3. Utilizadores 8
Figura 3 - Classe User e enumeração Role 8
2.1.4. Automações, Escalonamentos e Cenários 9
Figura 4 - Classes Automation, Scheduling e Scenario 10
2.1.5. Simulação do tempo 11
2.1.6. Sugestões Automáticas 11
Figura 5 - Motor de sugestões: InteractionRecord, Suggestion e SuggestionEngine 13
2.2. Interrogações 14
2.3. Exceções 14
Figura 6 - Conjunto de exceções personalizadas definidas no sistema 15
2.4. Persistência 16
3. Model-View-Controller **16**
3.1. Implementação do MVC 17
4. Descrição da Aplicação **18**
4.1. Gestão de Utilizadores 18
Figura 7 - Menu inicial da aplicação DomusControl 18
Figura 8 - Registo de novo utilizador com confirmação de sucesso 19
Figura 9 - Login com sucesso e apresentação do menu principal 19
4.2. Utilizador Administrador 20
Figura 10 - Menu principal do utilizador administrador 20
Figura 11 - Lista de casas associadas ao utilizador 20
Figura 12 - Criação de uma nova divisão 21
Figura 13 - Adição de um novo dispositivo a uma divisão 21
4.3. Utilizador Regular 22
Figura 14 - Menu principal do utilizador regular após login 22
Figura 15 - Erro de permissão ao tentar gerir divisões sem ser administrador 22
4.4. Operações sobre Dispositivos 23
Figura 16 - Menu de gestão de dispositivos 23
Figura 17 - Ligação de um dispositivo com confirmação de sucesso 23
Figura 18 - Consulta do estado detalhado de um dispositivo 24

Figura 19 - Lista de todos os dispositivos da casa organizados por divisão.....24
4.5. Automações e Escalonamentos.....25
Figura 20 - Menu de escalonamentos.....25
Figura 21 - Lista dos quatro escalonamentos definidos para a casa.....25
Figura 22 - Verificação dos escalonamentos para a hora atual de simulação.....26
4.6. Cenários.....26
Figura 23 - Menu de cenários.....26
Figura 24 - Lista dos quatro cenários do utilizador.....27
Figura 25 - Execução do cenário "Fora de Casa" com confirmação de sucesso.27
Figura 26 - Execução do cenário "Acordar" com confirmação de sucesso.....28
4.7. Estatísticas.....28
Figura 27 - Estatísticas de consumo e divisões com mais dispositivos.....28
Figura 28 - Top 3 dispositivos por número de ativações e por tempo ligado.....29
4.8. Gravação e Carregamento de Estado.....29
4.9. Sugestões Automáticas.....30
Figura 29 - Menu de sugestões automáticas com a lista de sugestões geradas.30
Figura 30 - Aceitação de uma sugestão e integração no sistema.....31
5. Estado de Teste.....31
6. Conclusão.....32
6.1. Principais Dificuldades.....32
6.2. Trabalho Futuro.....32
6.3. Aprendizagens.....32

***ANEXOS*.....33**
Diagrama UML Completo.....33

1. INTRODUÇÃO

O *DomusControl* é uma aplicação de automação doméstica desenvolvida em Java no âmbito da unidade curricular de Programação Orientada aos Objetos. Inspirada em plataformas como o Home Assistant, permite ao utilizador gerir casas inteligentes, organizando-as em divisões e associando-lhes dispositivos conectados de diferentes tipos. Sobre estes dispositivos é possível atuar de forma individual ou indiretamente, através de automações dependentes de sensores, escalonamentos baseados em horas do dia e cenários que executam ações coordenadas em múltiplos dispositivos.

O projeto foi desenvolvido de forma progressiva, acompanhando a estrutura modular definida no enunciado: gestão das entidades base, recolha de estatísticas e persistência em ficheiro, automações, escalonamentos e cenários, e sugestão automática de automações e escalonamentos com base no histórico de interações. Para além de cobrir estes requisitos, foi dada particular atenção ao respeito pelos princípios da programação orientada a objetos, à robustez do código e à clareza da camada de interação com o utilizador, implementada através de menus em modo texto.

Este relatório está organizado da seguinte forma. O **Capítulo 2** apresenta o diagrama de classes e justifica as decisões de design tomadas. O **Capítulo 3** descreve a arquitetura Model-View-Controller adotada. O **Capítulo 4** ilustra as funcionalidades da aplicação. O **Capítulo 5** descreve o estado de teste fornecido para a apresentação e, por fim, o **Capítulo 6** apresenta as conclusões, dificuldades encontradas e perspetivas de trabalho futuro.

2. Diagrama de Classes

Este capítulo apresenta a arquitetura de classes do sistema *DomusControl*. O diagrama de classes constitui uma parte central do design, oferecendo uma visão estrutural da aplicação ao descrever as classes que a compõem, os seus atributos e métodos, e as relações entre elas.

Para a construção do diagrama foi utilizada a ferramenta **Visual Paradigm**, uma plataforma gratuita de modelação com suporte completo à notação UML. Esta opção permitiu produzir um diagrama formal, com classes abstratas representadas em itálico, multiplicidades em ambas as extremidades das associações e estereótipos `«utility»` e `«enumeration»` devidamente assinalados. A versão integral do diagrama é apresentada em página própria no **Anexo A**.

2.1. Entidades da Aplicação

Esta secção descreve as hierarquias de classes utilizadas para representar as entidades centrais do sistema: dispositivos, casas, divisões, utilizadores, automações, escalonamentos, cenários e a representação interna do tempo.

2.1.1. Hierarquia de Dispositivos

A classe `Device` é abstrata e centraliza os atributos comuns a todos os dispositivos do sistema: identificador (`id`), `marca`, `modelo`, consumo nominal por hora (`consumoWh`) e estado de funcionamento (`ligado`). Para suportar as estatísticas exigidas no enunciado, inclui ainda três atributos adicionais: `totalMinutosLigado`, `numActivacoes` e `ultimaActivacao`, que são automaticamente atualizados pelos métodos `ligar(LocalDateTime agora)` e `desligar(LocalDateTime agora)`. Sempre que o dispositivo é ligado, `numActivacoes` é incrementado e o instante é guardado; ao desligar, é calculada a diferença em minutos relativamente à última ativação e somada ao tempo total. Esta lógica é executada uma única vez na superclasse, garantindo que qualquer dispositivo contabiliza corretamente as suas métricas sem duplicação de código.

Para garantir o polimorfismo, `Device` declara dois métodos abstratos: `getEstadoDetalhado()`, que devolve uma representação textual específica de cada tipo (incluindo, por exemplo, o brilho de uma lâmpada ou o grau de abertura de uma cortina), e `clone()`, que cria uma cópia da subclasse concreta. Esta abordagem dispensa o uso de `instanceof` na restante aplicação: a View pede simplesmente o estado detalhado e cada subclasse responde com a informação relevante.

Entre `Device` e as classes concretas existem duas classes abstratas intermédias que refletem comportamentos fundamentalmente distintos. `Actuator` representa dispositivos que **atuam** sobre o ambiente: ligam-se, desligam-se, abrem-se, fecham-se. `Sensor` representa dispositivos que **leem** valores do ambiente, expondo um método `getValorAtual()` cujo resultado é consumido pelas automações.

As subclasses concretas implementadas são: `Relay` e `Socket` (apenas ligam ou desligam), `Lamp` e `ColorLamp` (com controlo de brilho e, no segundo caso, temperatura de cor entre 2700K e 4000K), `Speaker` (com controlo de volume), `Curtain` e `GarageGate` (com grau de abertura entre 0% e 100%), `RainSensor` (pluviosidade) e `LuminositySensor` (luminosidade ambiente). Todas implementam **`Serializable`**, requisito necessário para a gravação em ficheiro binário descrita na Secção 2.4.

Esta arquitetura cumpre o requisito do enunciado segundo o qual a adição de novos tipos de dispositivos não deve obrigar a alterar a classe principal: basta criar uma nova subclasse de `Actuator` ou `Sensor` e implementar os dois métodos abstratos.

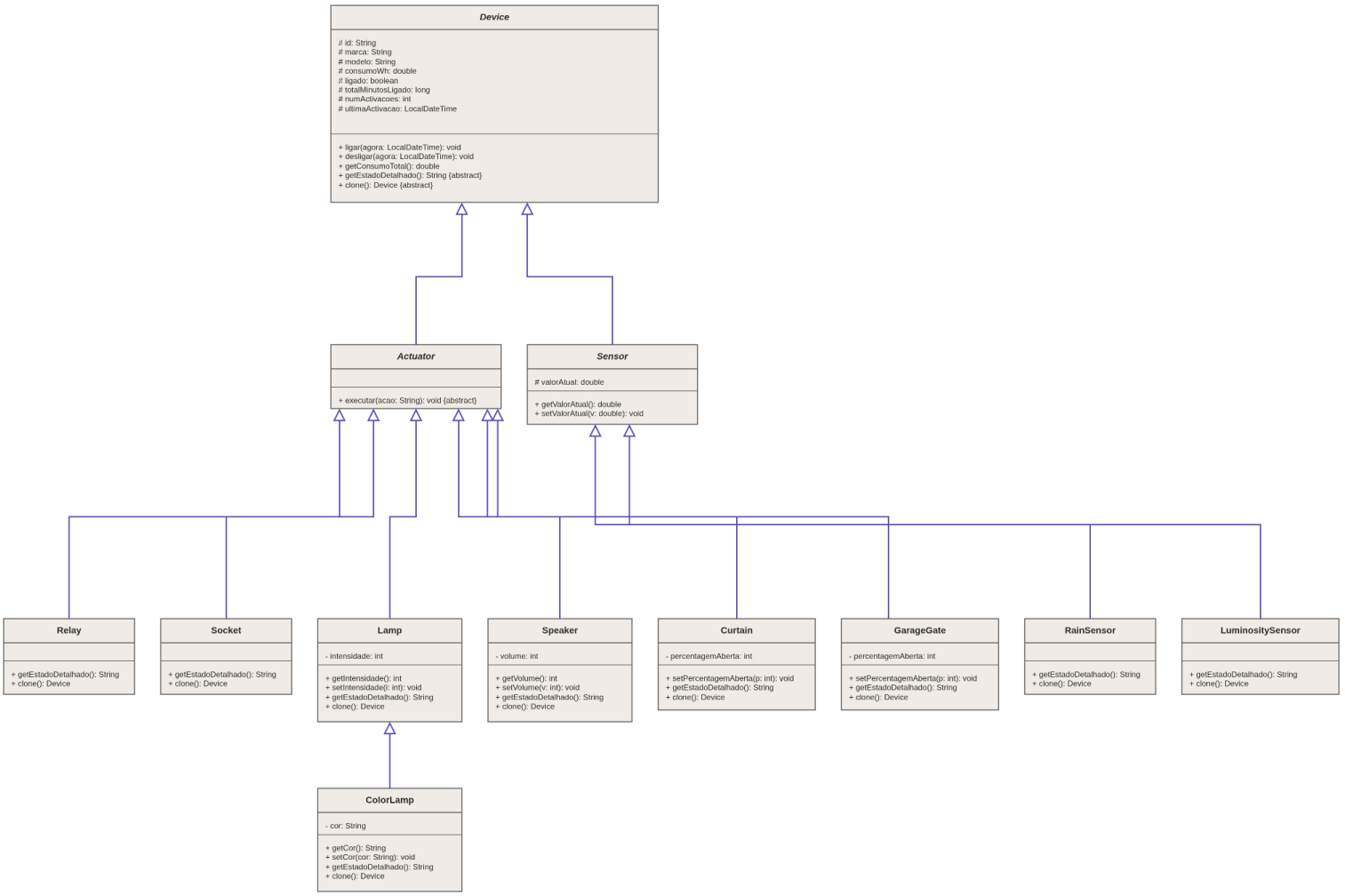


Figura 1 - *Hierarquia de classes da família `Device`*

2.1.2. Casas e Divisões

A classe `House` representa uma casa no sistema e é composta por uma lista de divisões (**`Room`**), uma lista de automações (**`Automation`**) e uma lista de escalonamentos (**`Scheduling`**), em que estes últimos associados à casa conforme indicado nas Secções 4 e 5 do enunciado. Cada `Room` mantém a lista dos dispositivos que lhe foram atribuídos.

A relação entre `House` e `Room` foi modelada como uma **composição**: uma divisão não tem existência autónoma fora da casa a que pertence. Esta decisão reflete-se no construtor de cópia de `House`, que percorre as divisões originais e cria novas instâncias de `Room`, garantindo que duas casas nunca partilham fisicamente a mesma divisão. Já a relação entre `Room` e `Device` é uma **agregação**: a divisão contém dispositivos, mas a operação sobre o estado destes (ligar, desligar, ler valores) tem de ocorrer no objeto real e não numa cópia, pelo que o controlador acede ao dispositivo através de uma referência interna (método privado `getDispositivoInterno`).

A classe `House` expõe ainda métodos de consulta agregados, em particular `getConsumoTotal()`, que soma o consumo de todos os dispositivos de todas as divisões e é utilizado pela estatística da casa que mais consome.

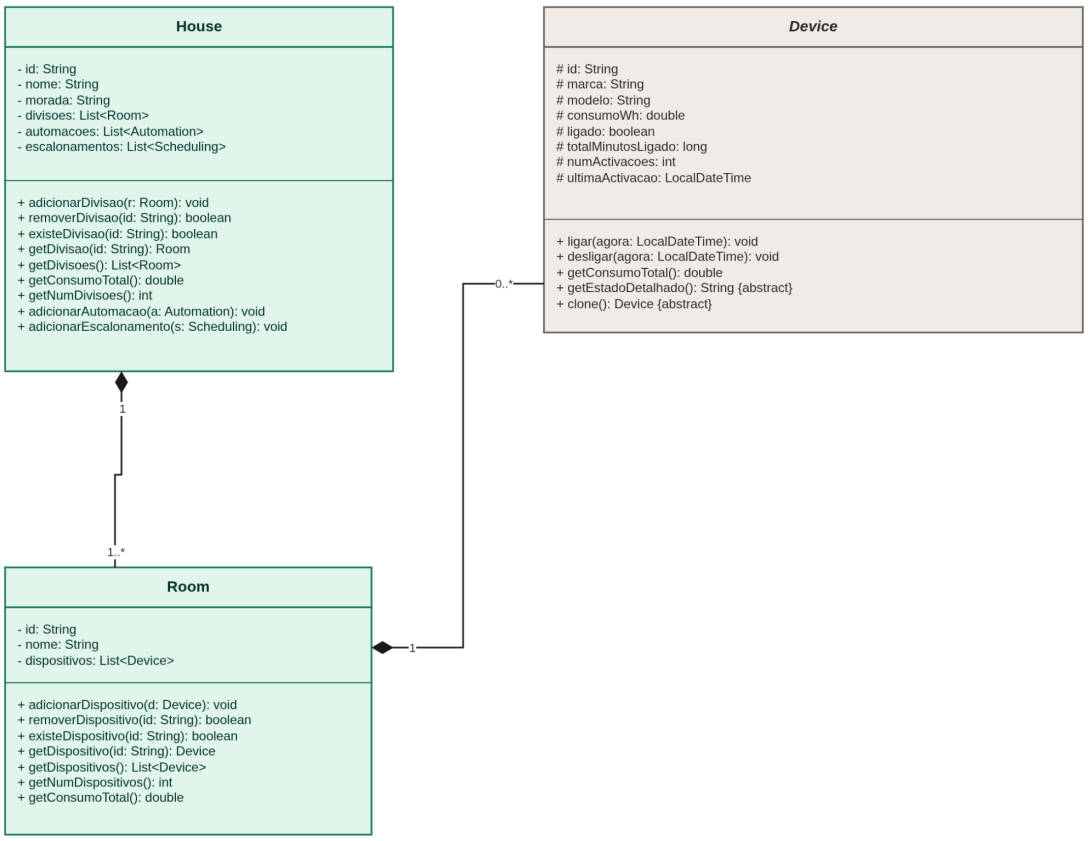


Figura 2 - *Relação entre `House`, `Room` e `Device`*

2.1.3. Utilizadores

Para representar os utilizadores foi tomada uma decisão de design importante. O enunciado refere explicitamente que um utilizador pode ser administrador numa casa e apenas utilizador noutra. Uma primeira abordagem, intuitiva, seria criar duas subclasses: `AdminUser` e `RegularUser`, mas isso tornaria impossível modelar exatamente este cenário, uma vez que o mesmo utilizador teria de pertencer simultaneamente a duas classes.

Por essa razão, optou-se por uma **única classe `User`** com um atributo `Map<String, Role> casas` que associa o identificador de cada casa ao papel que o utilizador detém nessa casa. Esta solução é mais flexível, evita duplicação de código e modela com fidelidade o requisito. O tipo `Role` é um `enum` com dois valores, `ADMIN` e `REGULAR`. A verificação do papel é feita através do método `isAdmin(String idCasa)`, invocado pelo controlador antes de qualquer operação que altere a estrutura da casa: quando o utilizador não tem o papel necessário, é lançada uma **`InvalidPermissionException`**.

Outra decisão relevante diz respeito aos cenários: como o enunciado refere na Secção 6 que *"cada utilizador pode definir os seus cenários"*, a lista de cenários (`List<Scenario>`) ficou **dentro do `User`** e não da `House`. Esta escolha respeita a semântica do enunciado e permite que dois utilizadores da mesma casa tenham cenários personalizados independentes.

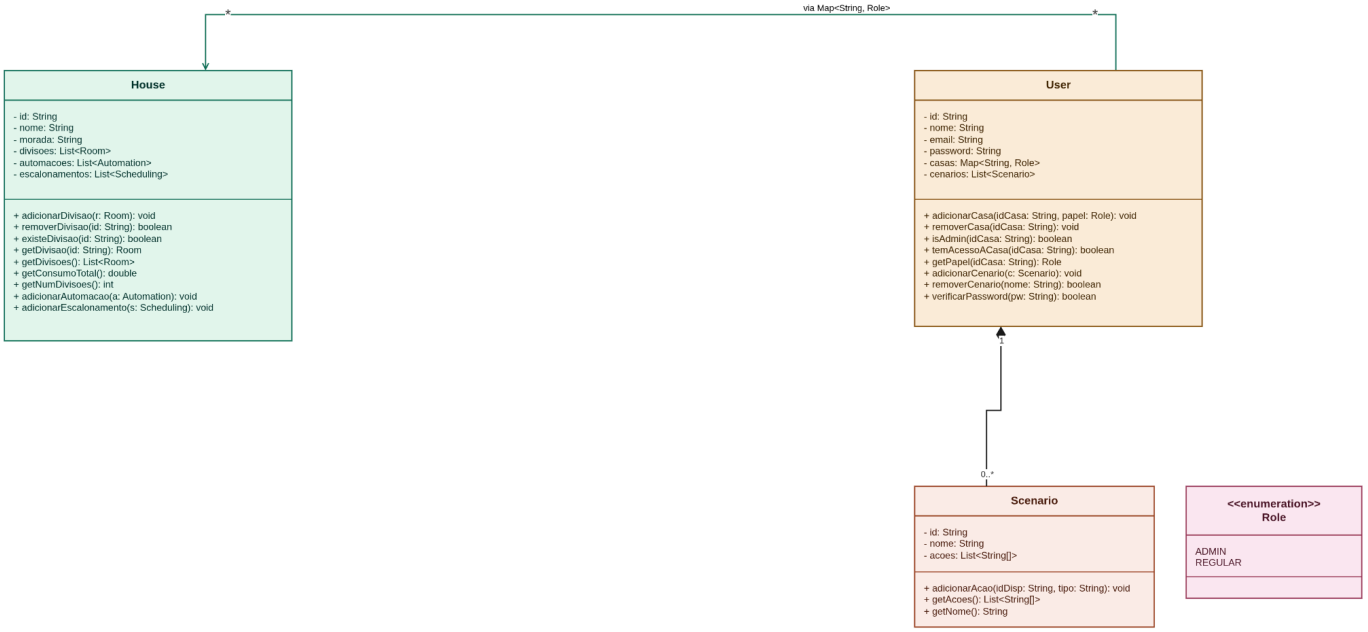


Figura 3 - *Classe `User` e enumeração `Role`*

2.1.4. Automações, Escalonamentos e Cenários

Estas três entidades são responsáveis pelo comportamento dinâmico do sistema, mas têm semânticas distintas que se refletem na sua estrutura e no local onde são guardadas.

Uma `Automation` é uma regra do tipo ***condição → ação***: associa um sensor (`idSensor`), um valor limiar (`limiar`) e uma ação (`"ligar"` ou `"desligar"`) que é aplicada a um conjunto de dispositivos quando a condição é avaliada como verdadeira. O método `avaliarCondicao(double valorSensor)` devolve `true` quando o valor lido pelo sensor atinge ou ultrapassa o limiar. As automações ficam **associadas à casa**, em coerência com a Secção 4 do enunciado, e são verificadas pelo controlador sempre que um sensor é atualizado.

Um `Scheduling` é uma variante temporal da automação: define uma `horaExecucao` (do tipo `LocalTime`) e uma ação a executar nessa hora. O método `deveExecutar(LocalTime horaAtual)` permite ao controlador determinar se o escalonamento deve disparar no instante atual da simulação. Tal como as automações, os escalonamentos ficam **associados à casa**, e o método `verificarEscalonamentos()` do controlador é invocado automaticamente sempre que o tempo é avançado.

Um `Scenario` representa uma sequência de ações a executar em vários dispositivos de forma coordenada. Internamente, mantém uma lista de pares `[idDispositivo, acao]`. Ao contrário das duas entidades anteriores, e conforme já referido em 2.1.3, os cenários ficam **associados ao utilizador**, refletindo o cariz pessoal e personalizável que o enunciado lhes atribui.

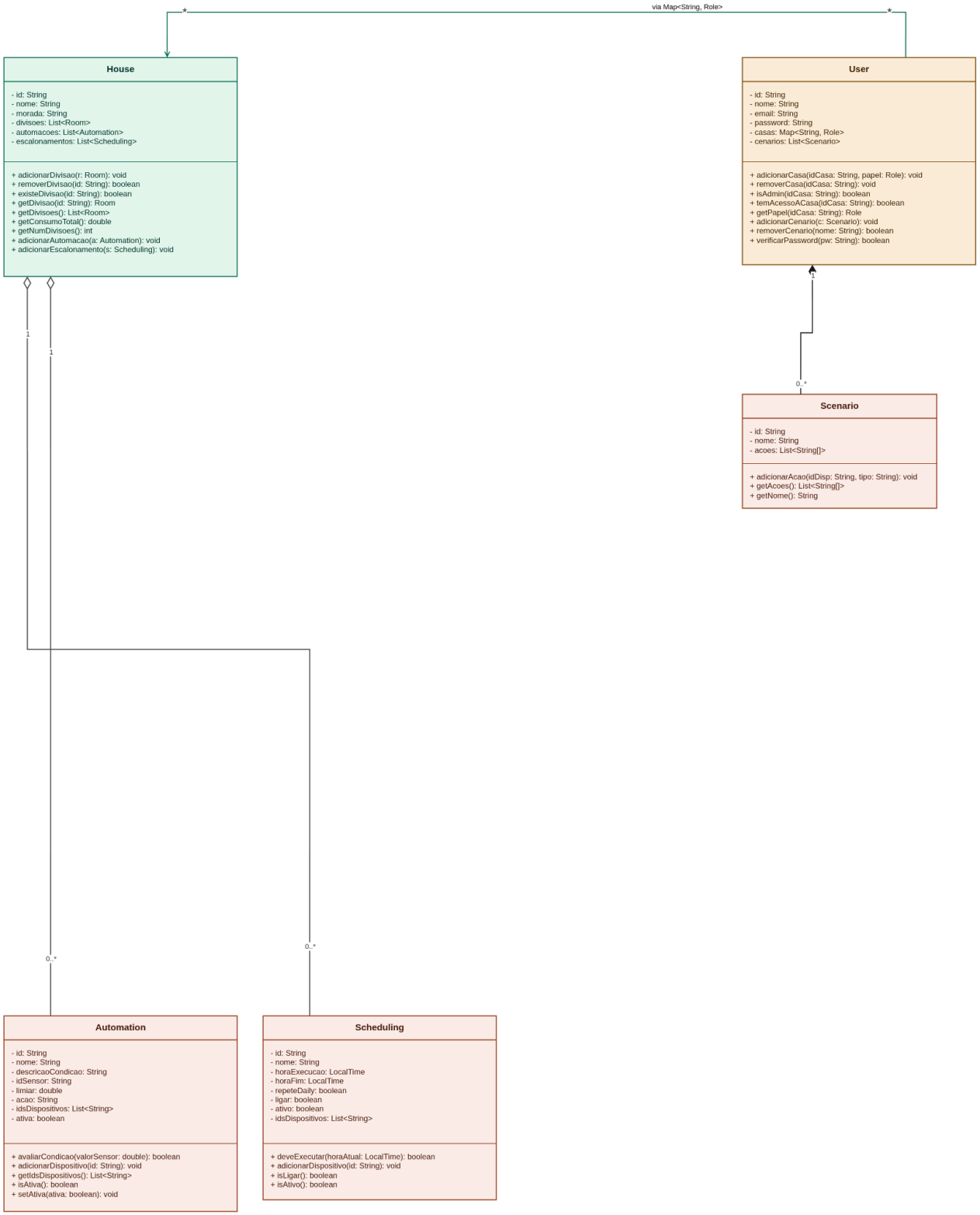


Figura 4 - *Classes `Automation`, `Scheduling` e `Scenario`*

2.1.5. Simulação do tempo

Como o enunciado refere explicitamente que a aplicação opera num ambiente de simulação, foi necessário criar uma representação interna de data/hora desligada do relógio real do sistema. Esta representação ficou encapsulada na classe `SimulatedClock`, que mantém um **`LocalDateTime`** atual e expõe métodos para o avançar manualmente: `avancarMinutos(long)`, `avancarHoras(long)` e `avancarDias(long)`. O controlador detém uma instância de `SimulatedClock` e é o único ponto da aplicação a partir do qual o tempo pode ser avançado, através do método **`avancarTempo(long minutos)`**, que avança o relógio e imediatamente verifica se algum escalonamento deve disparar.

A separação numa classe própria, em vez de manter o tempo simulado como variável dentro do controlador, torna explícito que o tempo é uma entidade do sistema e garante que qualquer parte do código que precise da hora atual a obtenha de forma uniforme através de `relogio.getDataHoraAtual()`.

2.1.6. Sugestões Automáticas

A aplicação inclui uma funcionalidade adicional que, com base nas interações passadas do utilizador, sugere automações ou escalonamentos potencialmente úteis. Esta funcionalidade foi implementada através de três classes que cooperam entre si: **`InteractionRecord`**, **`Suggestion`** e **`SuggestionEngine`**.

A classe `InteractionRecord` representa um registo de uma interação relevante e guarda toda a informação necessária para análise posterior: `idUtilizador`, `idDispositivo`, `nomeDispositivo`, `idCasa`, `tipoAcao` (`"ligar"`, `"desligar"` ou `"sensor"`), `dataHora` e, no caso de leituras de sensores, o `valorSensor`. A classe oferece dois construtores distintos: um para ações sobre dispositivos, outro para leituras de sensores, tornando assim explícita a diferença entre os dois tipos de evento. O controlador adiciona automaticamente um novo registo ao histórico (`List<InteractionRecord>`) sempre que um utilizador autenticado liga ou desliga um dispositivo, ou sempre que um sensor é atualizado.

A classe `Suggestion` representa uma sugestão gerada pelo motor de análise. Pode ser de dois tipos, distinguidos pelo enum interno `Tipo` (`ESCALONAMENTO` ou `AUTOMACAO`), e contém uma descrição textual em linguagem natural (mostrada ao utilizador) e a referência ao objeto `Scheduling` ou `Automation` que será adicionado ao sistema caso a sugestão seja aceite. Para garantir a consistência interna, optou-se pelo padrão *factory method*: o construtor é privado, e a criação só pode ser feita através dos métodos estáticos

de `Escalonamento(...)` e `deAutomacao(...)`, impedindo a criação de sugestões em estado inválido.

O núcleo desta funcionalidade é a classe `SuggestionEngine`, marcada com o estereótipo **«utility»** por possuir apenas métodos estáticos e um construtor privado. O método público `gerarSugestoes(historico, idCasa)` filtra os registos relevantes para a casa indicada e aplica dois algoritmos de deteção de padrões:

- **Padrão temporal (geração de escalonamentos):** os registos de tipo `"ligar"` são agrupados pela chave composta `(idDispositivo, hora-do-dia)`. Sempre que um par é observado em pelo menos `MIN_OCORRENCIAS = 3` ocasiões distintas, é gerada uma sugestão de escalonamento diário para essa hora. Por exemplo, se o utilizador ligar a lâmpada da sala às 19h em três dias, o sistema sugere a criação de um escalonamento *"Ligar lâmpada da sala às 19:00, todos os dias"*.

- **Padrão sensor → atuador (geração de automações):** para cada par possível `(sensor, dispositivo)`, o motor verifica quantas vezes uma leitura do sensor foi seguida, em menos de cinco minutos, pela ativação manual desse dispositivo. Se essa correlação ocorrer em pelo menos três ocasiões, é gerada uma sugestão de automação cujo limiar é a média dos valores observados nessas ocasiões.

Quando o utilizador aceita uma sugestão, o método `aceitarSugestao(idCasa, s)` do controlador adiciona o `Scheduling` ou a `Automation` correspondente à casa, integrando-a no funcionamento normal do sistema sem necessidade de qualquer reconfiguração adicional.

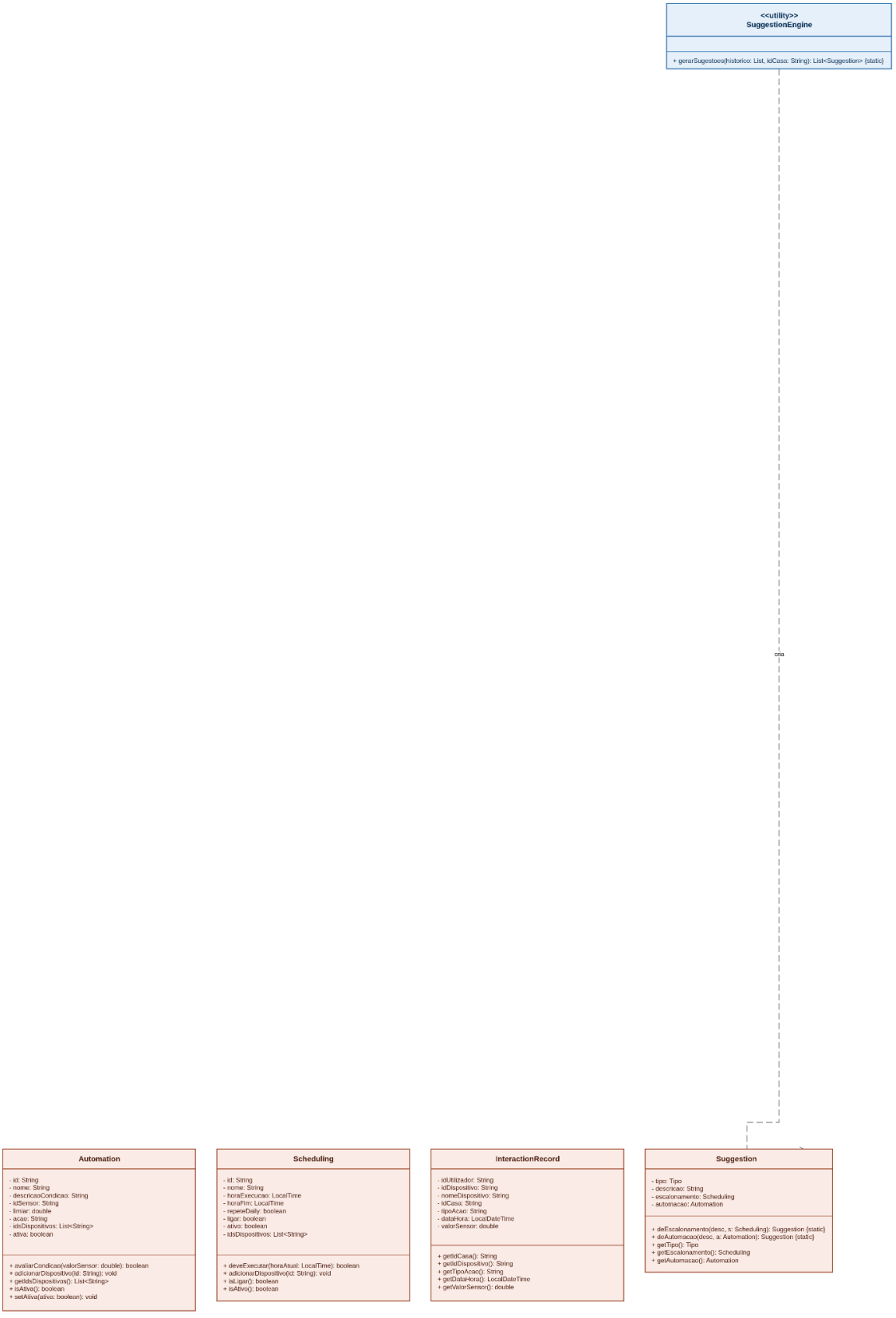


Figura 5 *- Motor de sugestões:* `InteractionRecord`*,* `Suggestion` *e* `SuggestionEngine`

2.2. Interrogações

A aplicação inclui um conjunto de interrogações que fornecem estatísticas e informações relevantes sobre a utilização do sistema, conforme exigido pelo enunciado (Secção 8.2). Estas interrogações estão implementadas na classe `DomusController`.

Entre as interrogações disponíveis destacam-se:

- **`getCasaQueMaisConsome()`**: devolve a casa com maior consumo energético total.
- **`getTop3DispositivosPorActivacoes(idCasa)`**: devolve os três dispositivos mais usados de uma casa em número de ativações.
- **`getTop3DispositivosPorTempo(idCasa)`**: devolve os três dispositivos mais usados de uma casa em tempo total ligado.
- **`getTop3DivisoesPorDispositivos()`**: devolve as três divisões com mais dispositivos em todo o sistema.
- **`getConsumoTotalDaCasa(idCasa)`**: método adicional que devolve o consumo total de uma casa específica.

A escolha de centralizar estas interrogações no controlador, em vez de as distribuir pelas entidades individuais, segue o princípio da **separação de responsabilidades**. As classes do modelo têm como responsabilidade representar fielmente o estado do domínio e expor apenas as operações que lhes são inerentes, como uma `House` sabe calcular o seu próprio consumo, mas não tem por que se comparar com outras casas; um `Device` sabe contabilizar as suas próprias ativações, mas não tem por que se ordenar relativamente aos seus pares. Já o controlador, por ser o ponto único onde existem referências para todas as entidades em simultâneo, é o local natural para implementar interrogações agregadas. Esta organização torna ainda o modelo independente do tipo de estatísticas pretendidas: alterar o cálculo ou acrescentar novas estatísticas exige apenas modificar o controlador.

2.3. Exceções

A aplicação define um conjunto de exceções personalizadas que sinalizam situações específicas em que a execução normal do programa não pode prosseguir. A criação destas exceções é importante para garantir a robustez e a clareza do código, permitindo lidar de forma controlada com casos inesperados.

As exceções definidas são:

- **`DeviceNotFoundException`**: lançada quando se tenta aceder a um dispositivo que não existe.
- **`RoomNotFoundException`**: lançada quando se tenta aceder a uma divisão que não existe.
- **`HouseNotFoundException`**: lançada quando se tenta aceder a uma casa que não existe.
- **`UserNotFoundException`**: lançada quando se tenta aceder a um utilizador que não existe, ou quando nenhuma sessão está iniciada.
- **`InvalidPermissionException`**: lançada quando um utilizador tenta realizar uma ação para a qual não tem permissão (tipicamente, alterar a estrutura de uma casa sem ser administrador).

A escolha de criar exceções específicas, em vez de reutilizar exceções genéricas como **`RuntimeException`** ou **`IllegalArgumentException`**, permite que a camada de View distinga, pelo tipo de exceção apanhada, a natureza do erro e apresente uma mensagem contextualizada ao utilizador. Algo que seria impossível com exceções genéricas. Esta abordagem melhora a legibilidade do código, separa claramente a lógica normal da lógica de erro e segue as boas práticas de programação orientada a objetos no que respeita ao tratamento formal de erros.

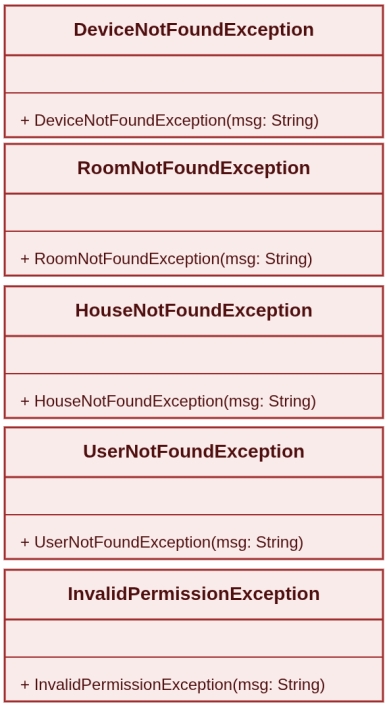


Figura 6 *- Conjunto de exceções personalizadas definidas no sistema.*

2.4. Persistência

A continuidade da experiência do utilizador entre sessões é garantida por um mecanismo de persistência baseado na **serialização nativa de objetos Java**. Para que tal seja possível, todas as classes do modelo, sem exceção, implementam a interface **`Serializable`** e declaram um `serialVersionUID`, garantindo a compatibilidade entre o estado gravado e o estado recarregado.

A operação é exposta pelo `DomusController` através de dois métodos. O método `gravarEstado(String caminho)` serializa o próprio controlador (este que mantém referências para todas as casas, todos os utilizadores, o relógio simulado e o histórico de interações) para um ficheiro binário, recorrendo a um `ObjectOutputStream` envolto num `FileOutputStream`. O método estático `carregarEstado(String caminho)` faz a operação inversa, lendo o ficheiro com um `ObjectInputStream` e devolvendo uma nova instância de `DomusController` com o estado integralmente restaurado.

A escolha de ficheiros binários, em vez de formatos textuais como JSON ou XML, decorre diretamente do enunciado (Secção 10), mas tem também justificações técnicas: a serialização nativa preserva automaticamente a hierarquia de classes (incluindo o tipo concreto de cada `Device`, algo que exigiria código adicional num formato textual), é mais eficiente em espaço e em tempo, e elimina a necessidade de escrever manualmente código de leitura e escrita para cada tipo de objeto.

3. Model-View-Controller

O padrão arquitetural Model-View-Controller (MVC) é uma das abordagens mais utilizadas no desenvolvimento de aplicações com interface, e promove a separação de responsabilidades entre três componentes distintos: a *Model*, que representa os dados e a lógica de negócio; a *View*, responsável pela interação com o utilizador; e o *Controller*, que coordena o fluxo entre as duas anteriores.

Esta separação traz vantagens concretas: facilita a manutenção, porque alterações na apresentação não obrigam a tocar no modelo (e vice-versa); facilita a evolução, porque novas funcionalidades podem ser introduzidas em cada camada com impacto controlado nas restantes; e clarifica a leitura do código, porque cada classe tem uma responsabilidade bem definida. No projeto DomusControl, o MVC foi adotado desde o início, refletindo-se diretamente na organização dos packages e na forma como as classes comunicam entre si.

3.1. Implementação do MVC

Cada uma das três camadas do MVC tem responsabilidades bem delimitadas no *DomusControl*.

A camada **Model** concentra todas as entidades do domínio e a lógica que lhes é própria. Está organizada nos packages `model.devices` (hierarquia de dispositivos), `model.house` (casas e divisões), `model.users` (utilizadores e papéis), `model.automations` (automações, escalonamentos, cenários, registos de interação e sugestões) e `utils` (relógio simulado, dados de teste, motor de sugestões e gestão de cores no terminal). Nenhuma destas classes comunica diretamente com o utilizador (não imprimem mensagens nem leem do `Scanner`) o que garante que o modelo é independente do tipo de interface utilizada e poderia, no futuro, ser reutilizado com uma interface gráfica sem qualquer alteração.

A camada **View**, no package `view`, é composta por menus em modo texto, cada um responsável por um subdomínio funcional: `MainMenu` para autenticação e registo, `HouseMenu` para gestão de casas e divisões, `DeviceMenu` para operação sobre dispositivos, `ScenarioMenu` para cenários, `SchedulingMenu` para escalonamentos, `AutomationMenu` para automações e `SuggestionMenu` para sugestões automáticas. A View nunca acede diretamente ao modelo: todas as operações são canalizadas através do controlador, e qualquer alteração do estado da aplicação tem obrigatoriamente de passar pelos seus métodos públicos.

A camada **Controller**, materializada na classe `DomusController` no package `controller`, é o intermediário entre a View e o Model. Recebe os comandos da View, valida-os (incluindo as verificações de permissão, descritas em 2.1.3), invoca os métodos adequados nas entidades do modelo, contabiliza as interações no histórico que alimenta o motor de sugestões e devolve o resultado à View, isto é, tipicamente sob a forma de cópias defensivas dos objetos do modelo, evitando que a View possa alterar acidentalmente o estado interno.

Esta separação trouxe vantagens concretas ao longo do desenvolvimento. Permitiu alterar várias vezes a apresentação visual dos menus (cores, banners, organização) sem tocar numa única linha do modelo; permitiu testar a lógica de negócio diretamente, instanciando o controlador em código sem necessidade de simular interação textual; e tornou natural a integração da funcionalidade de sugestões automáticas, que se limitou a adicionar três novas classes ao modelo, um novo menu à View e dois métodos ao controlador, sem impacto no que já existia.

4. Descrição da Aplicação

A aplicação *DomusControl* foi concebida como uma plataforma de automação doméstica, onde os utilizadores podem criar contas, gerir as suas casas e divisões, associar dispositivos e controlá-los de forma individual ou através de automações, escalonamentos e cenários. Toda a interação com o utilizador é realizada através de menus textuais, organizados e apresentados pela camada View.

4.1. Gestão de Utilizadores

A gestão de utilizadores é feita através do **MainMenu**, que permite registar novos utilizadores e autenticar utilizadores existentes. O registo solicita id, nome, email e password. A autenticação verifica as credenciais e inicia a sessão. Cada utilizador tem associado um mapa de casas com o seu papel (ADMIN ou REGULAR). Os cenários pertencem ao utilizador e não à casa.

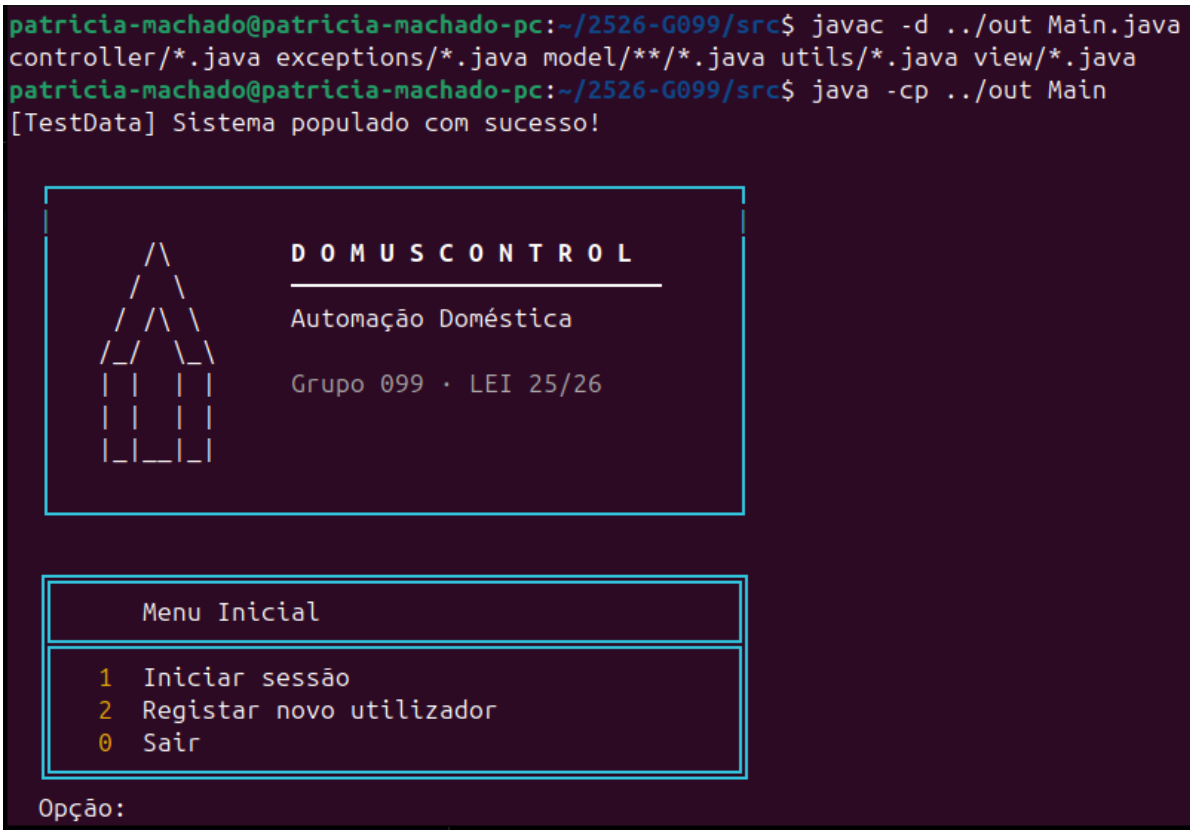


Figura 7 - *Menu inicial da aplicação DomusControl*

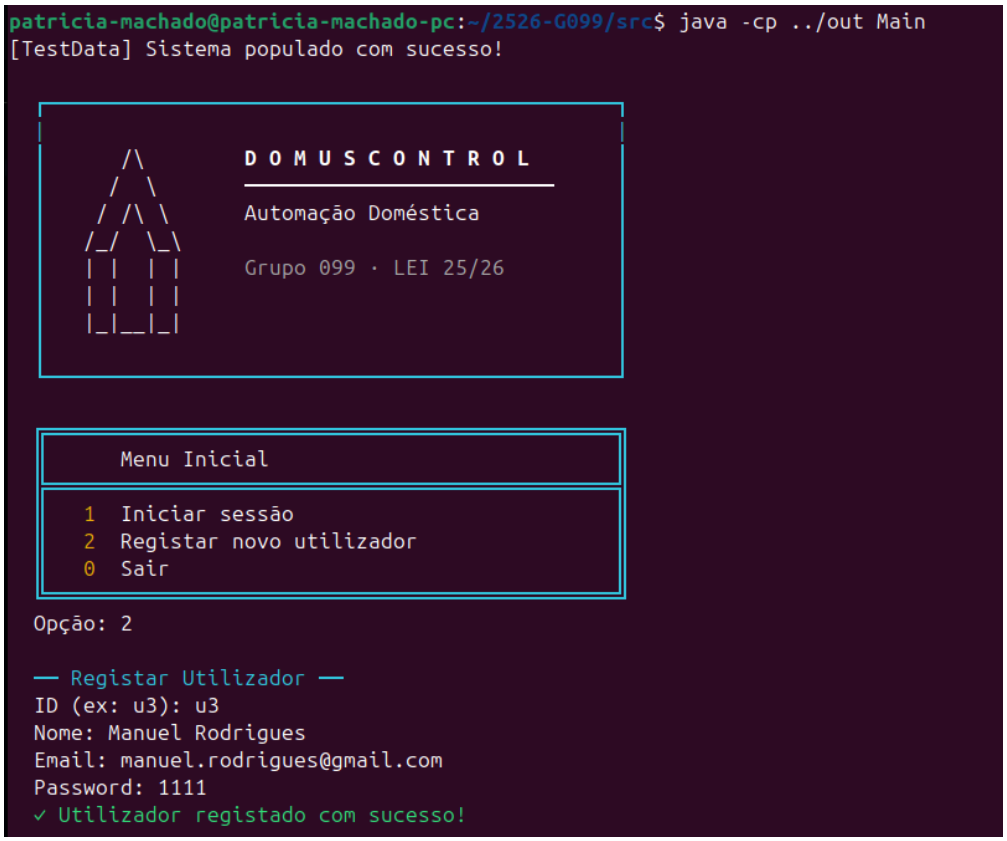


Figura 8 - *Registo de novo utilizador com confirmação de sucesso*

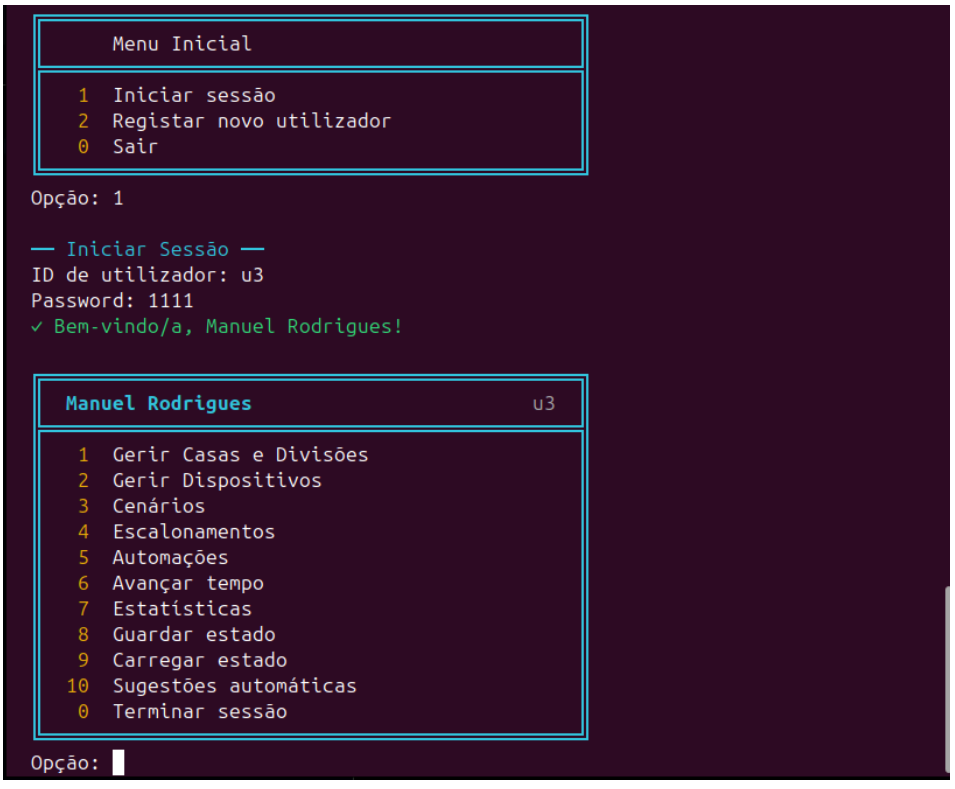


Figura 9 - *Login com sucesso e apresentação do menu principal*

4.2. Utilizador Administrador

Os utilizadores com papel de administrador numa casa têm permissão para criar divisões, adicionar dispositivos e configurar o sistema. São também responsáveis pela criação inicial da casa.

O administrador acede ao HouseMenu onde pode criar casas, adicionar divisões e associar dispositivos. Ao criar uma casa, o utilizador fica automaticamente como ADMIN. Só administradores podem alterar a estrutura da casa - caso contrário é lançada InvalidPermissionException.

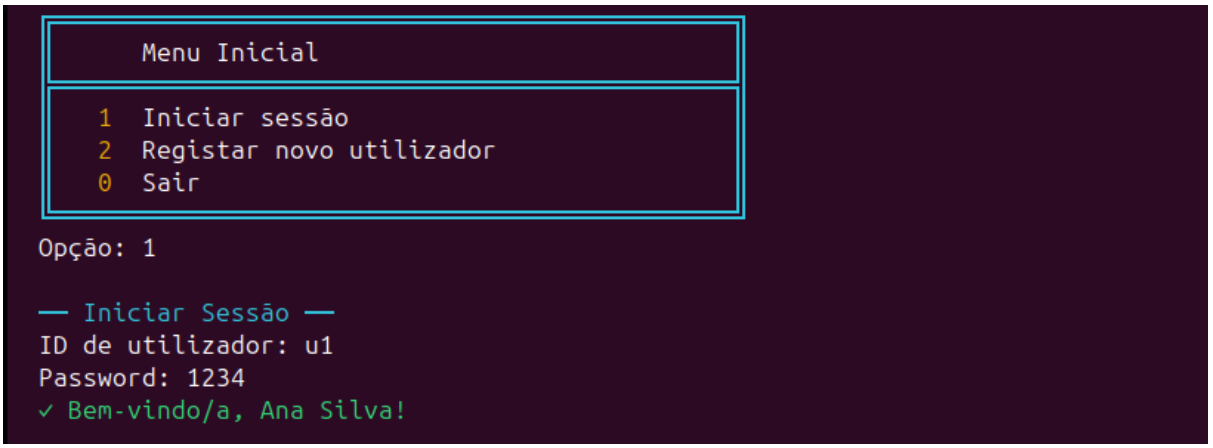


Figura 10 - *Menu principal do utilizador administrador*

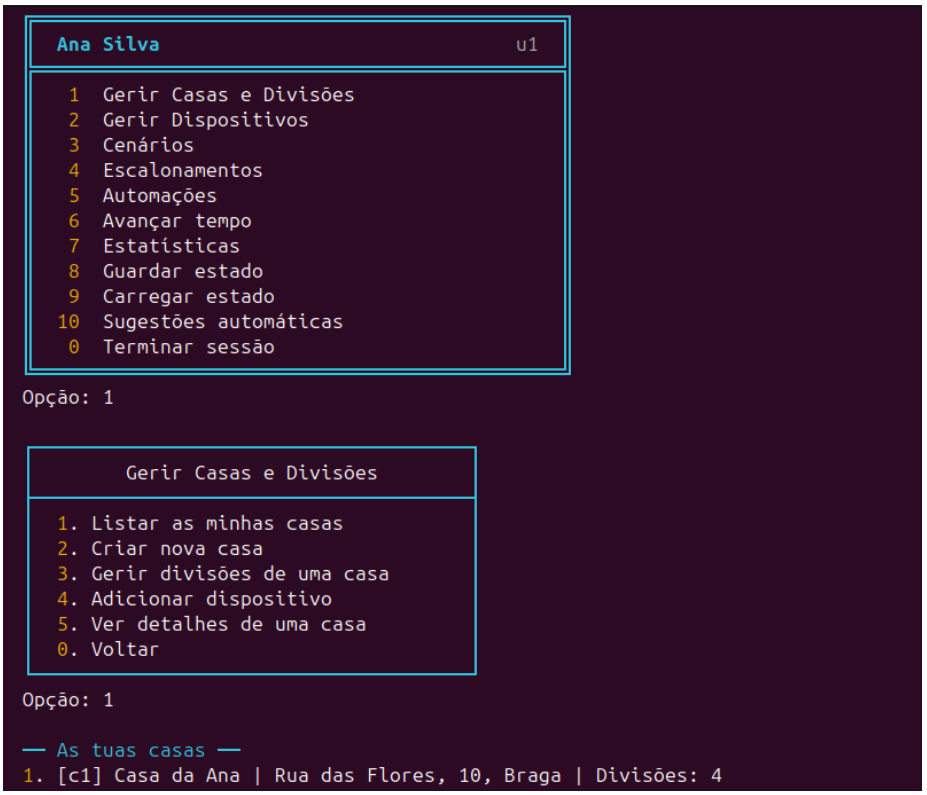


Figura 11 - *Lista de casas associadas ao utilizador*

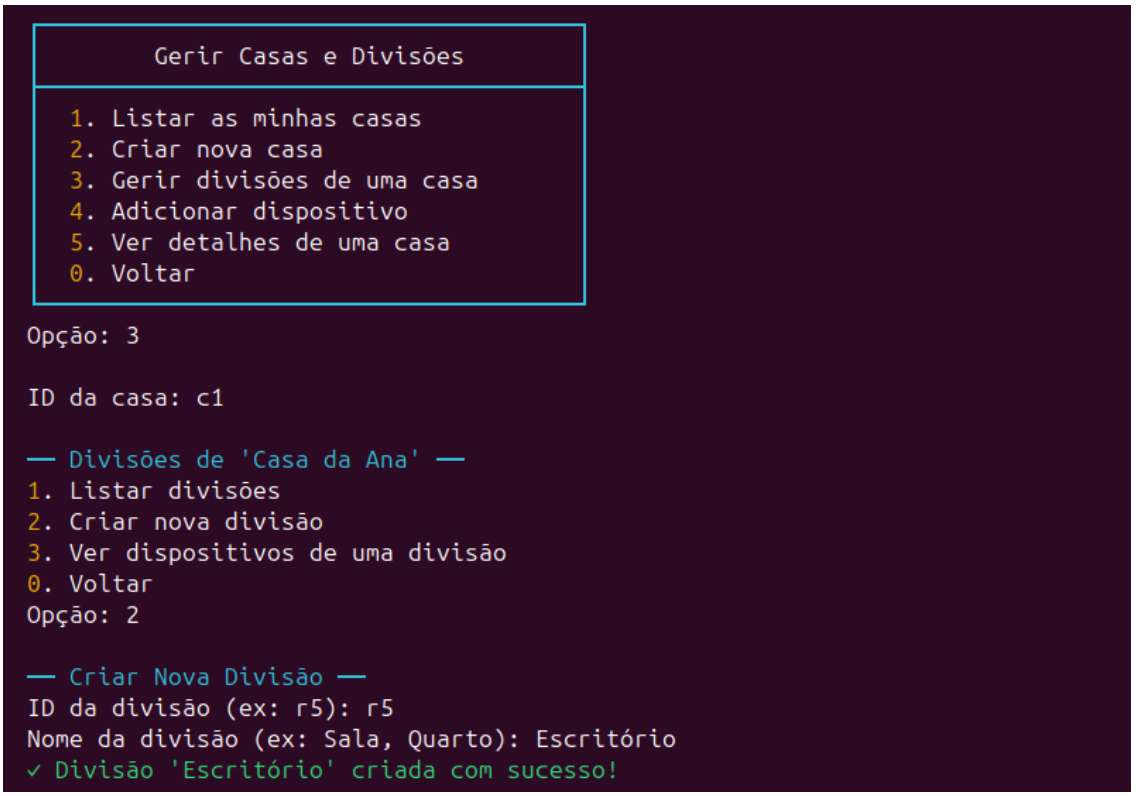


Figura 12 - *Criação de uma nova divisão*

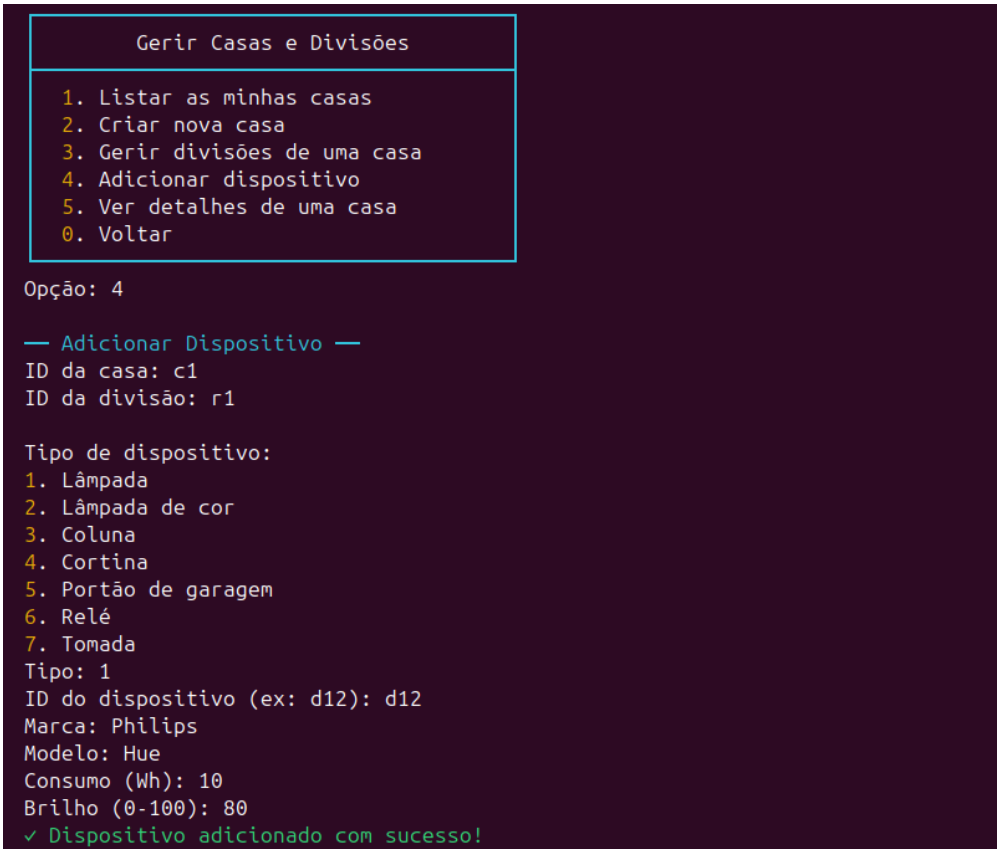


Figura 13 - *Adição de um novo dispositivo a uma divisão*

4.3. Utilizador Regular

Os utilizadores regulares podem operar os dispositivos existentes, invocar cenários, espoletar automações e escalonamentos. No entanto, não têm permissão para alterar a estrutura da casa.

O utilizador regular pode iniciar sessão, ver as casas a que tem acesso, operar dispositivos individualmente, executar cenários e verificar escalonamentos. Não tem acesso às opções de criação de casas, divisões ou dispositivos.

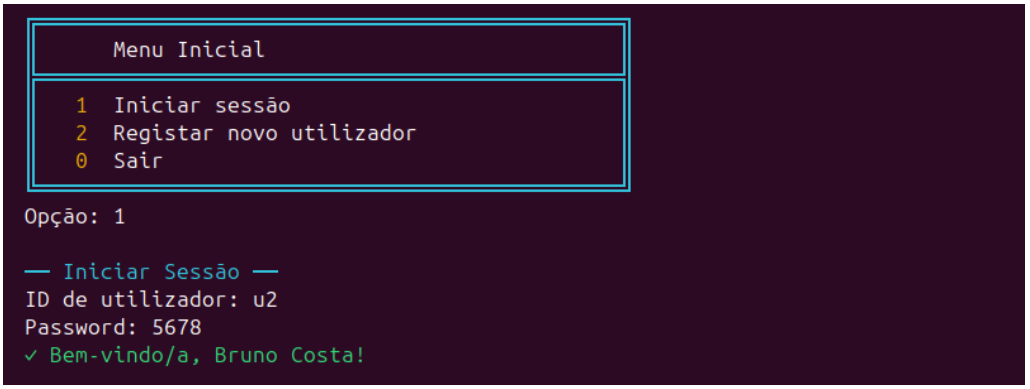


Figura 14 - *Menu principal do utilizador regular após login*



Figura 15 - *Erro de permissão ao tentar gerir divisões sem ser administrador*

4.4. Operações sobre Dispositivos

Os dispositivos são operados através do **DeviceMenu**, que permite ligar, desligar e consultar o estado de qualquer dispositivo. O sistema suporta 7 tipos: Lamp, ColorLamp, Speaker, Curtain, GarageGate, Relay e Socket. Cada tipo apresenta o seu estado detalhado de forma polimórfica através do método **getEstadoDetalhado()**.

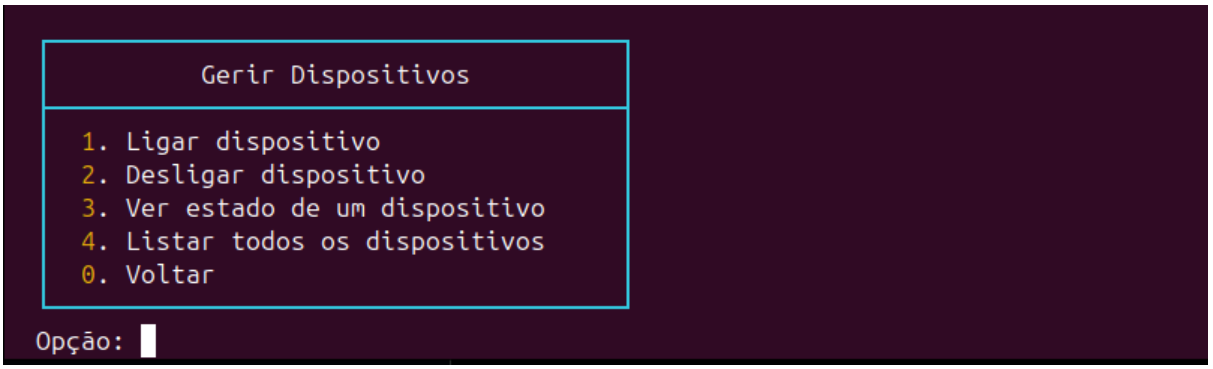


Figura 16 - *Menu de gestão de dispositivos*

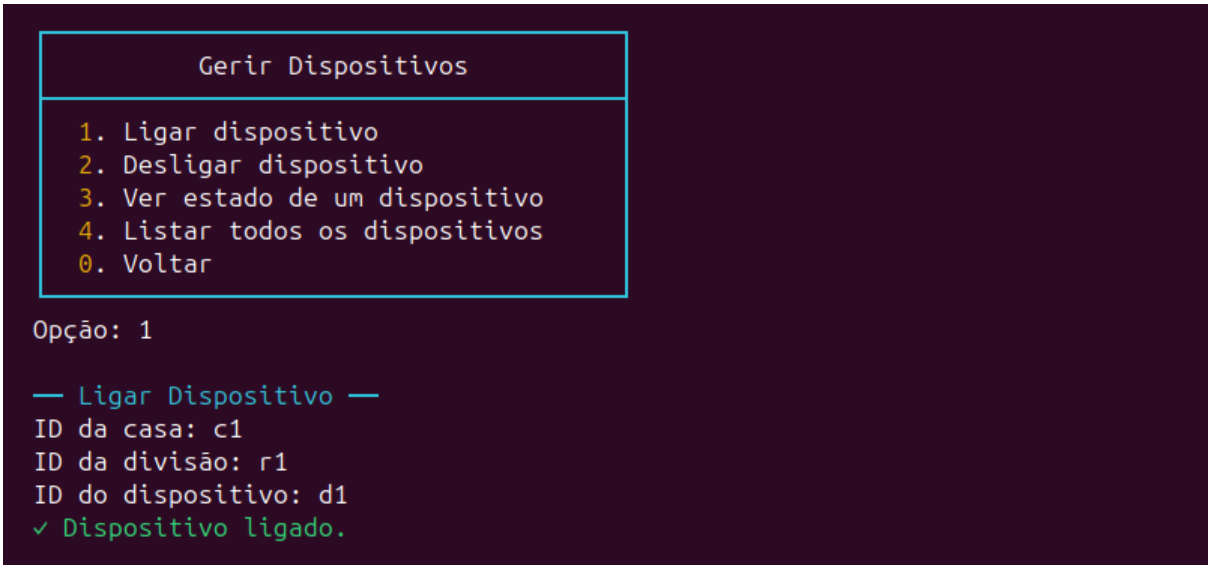


Figura 17 - *Ligação de um dispositivo com confirmação de sucesso*



Figura 18 - *Consulta do estado detalhado de um dispositivo*

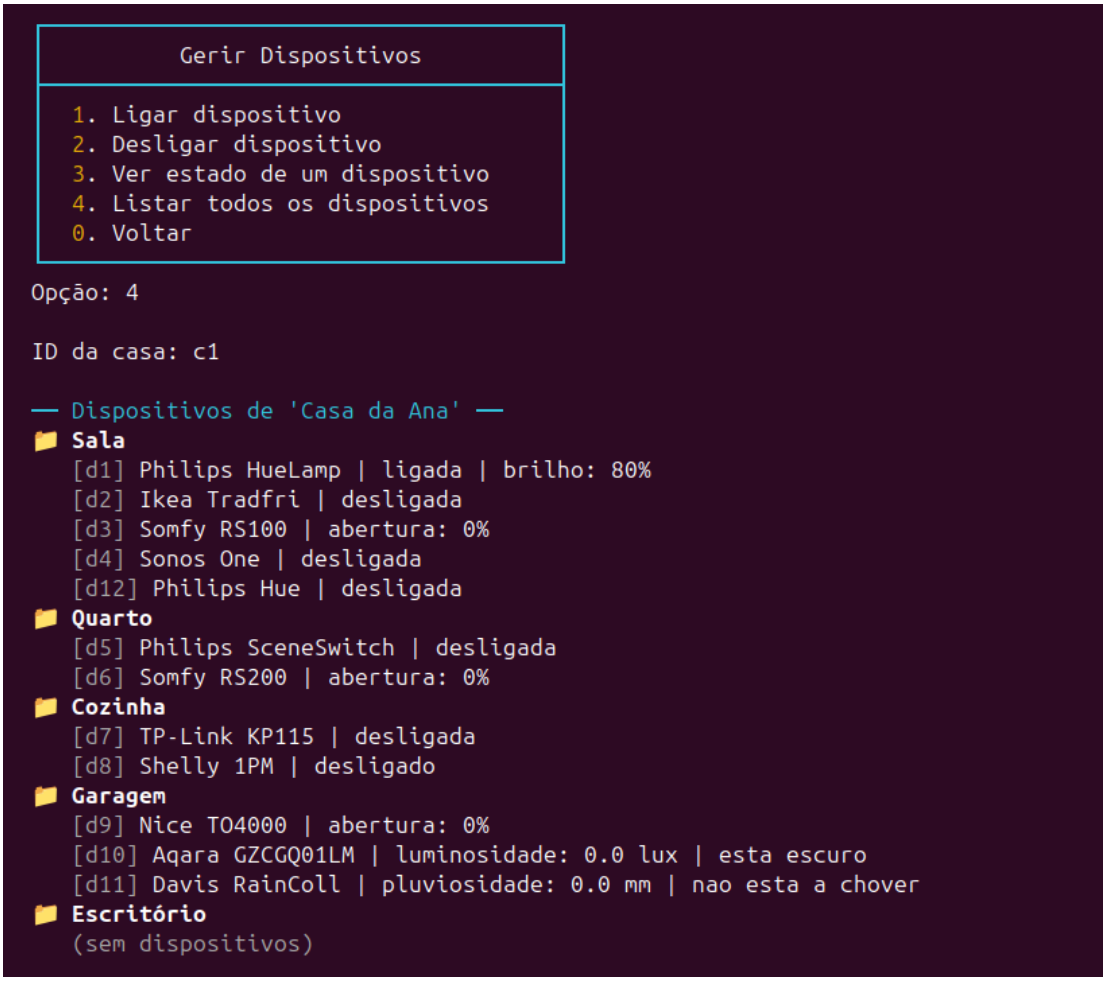


Figura 19 - *Lista de todos os dispositivos da casa organizados por divisão*

4.5. Automações e Escalonamentos

Os escalonamentos são geridos através do **`SchedulingMenu`**. Foram definidos 4 escalonamentos no estado de teste: abrir cortinas às 07:30, fechar cortinas às 21:00, ligar luzes às 19:00 e desligar tudo à meia-noite. Cada escalonamento define uma hora de execução, uma ação (ligar/desligar) e os dispositivos afetados. A verificação é feita através do método **`verificarEscalonamentos()`** do controller.

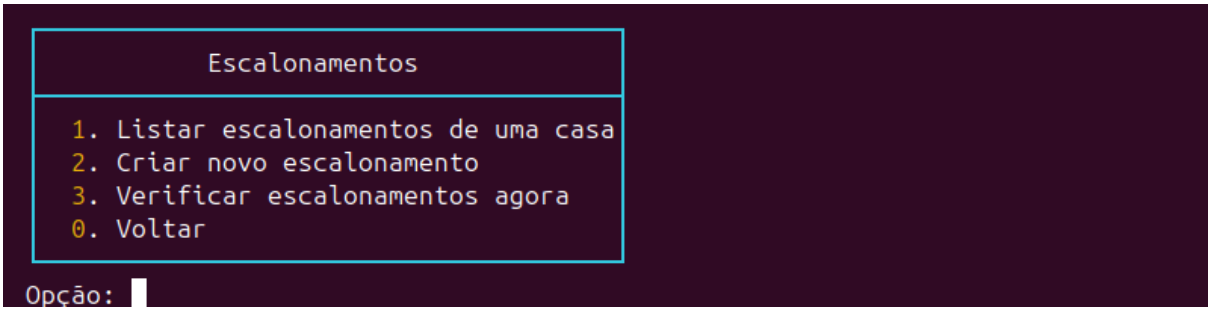


Figura 20 - *Menu de escalonamentos*

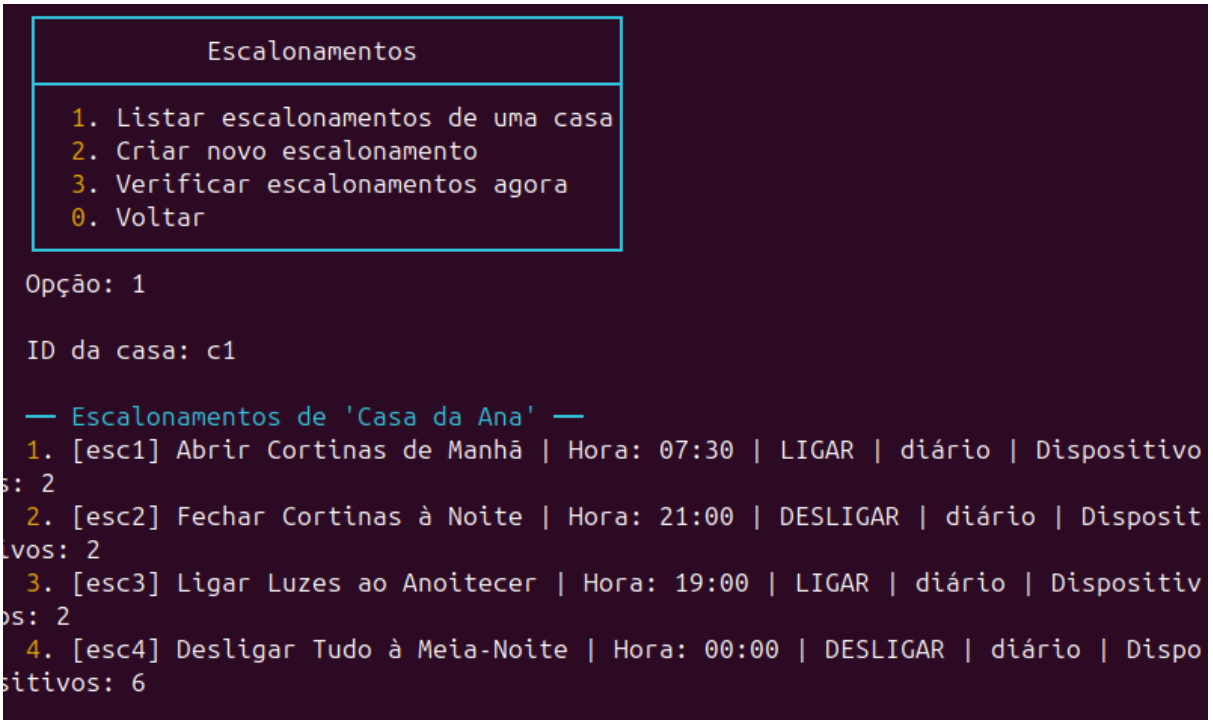


Figura 21 - *Lista dos quatro escalonamentos definidos para a casa*

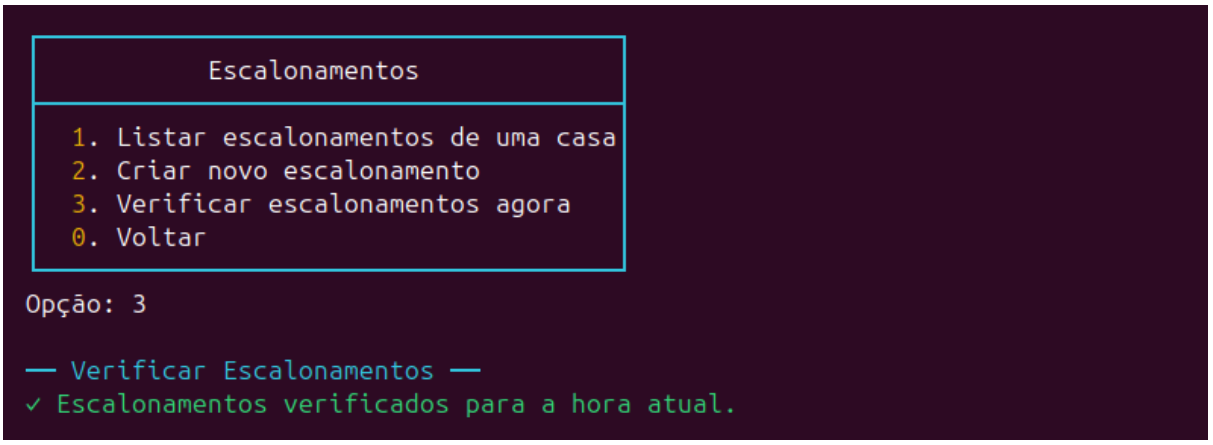


Figura 22 - *Verificação dos escalonamentos para a hora atual de simulação*

4.6. Cenários

Os cenários permitem executar um conjunto de ações sobre vários dispositivos ao mesmo tempo. Foram implementados os quatro cenários obrigatórios indicados no enunciado:

- **Fora de Casa:** desliga todas as luzes, fecha todas as cortinas e desliga todas as colunas de som
- **Jantar com Amigos:** ajusta a luminosidade e o som para uma refeição com convidados
- **Deitar:** desliga as luzes principais e baixa as cortinas
- **Acordar:** abre as cortinas e liga as luzes gradualmente

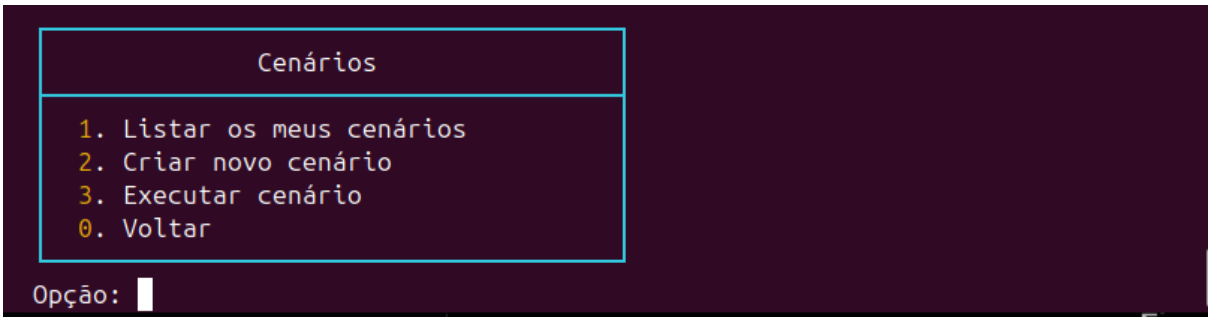


Figura 23 - *Menu de cenários*

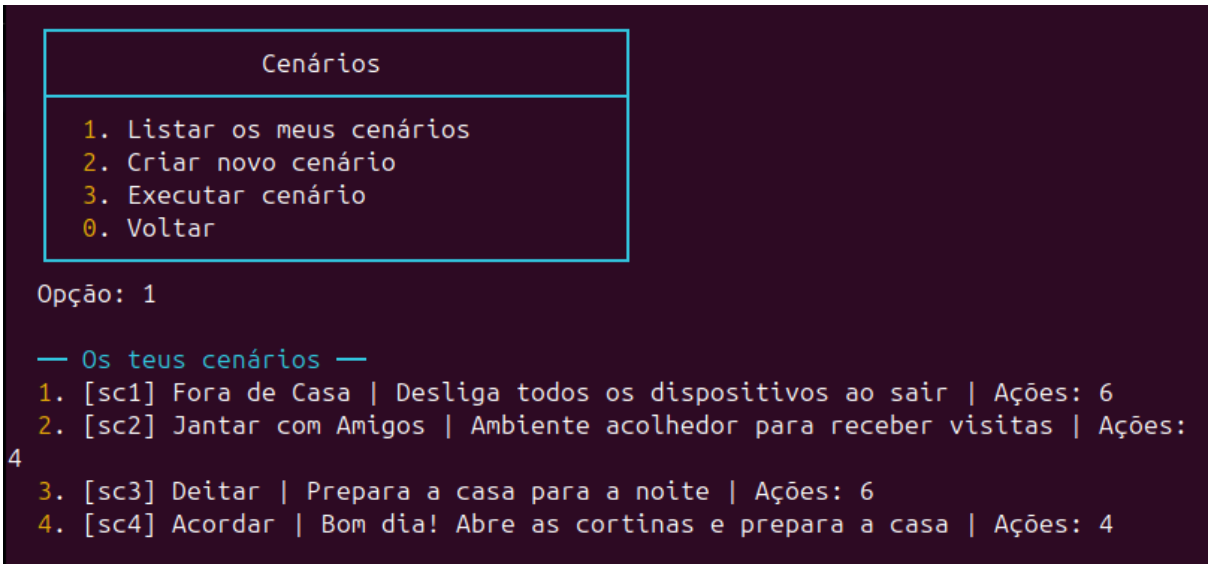


Figura 24 - *Lista dos quatro cenários do utilizador*

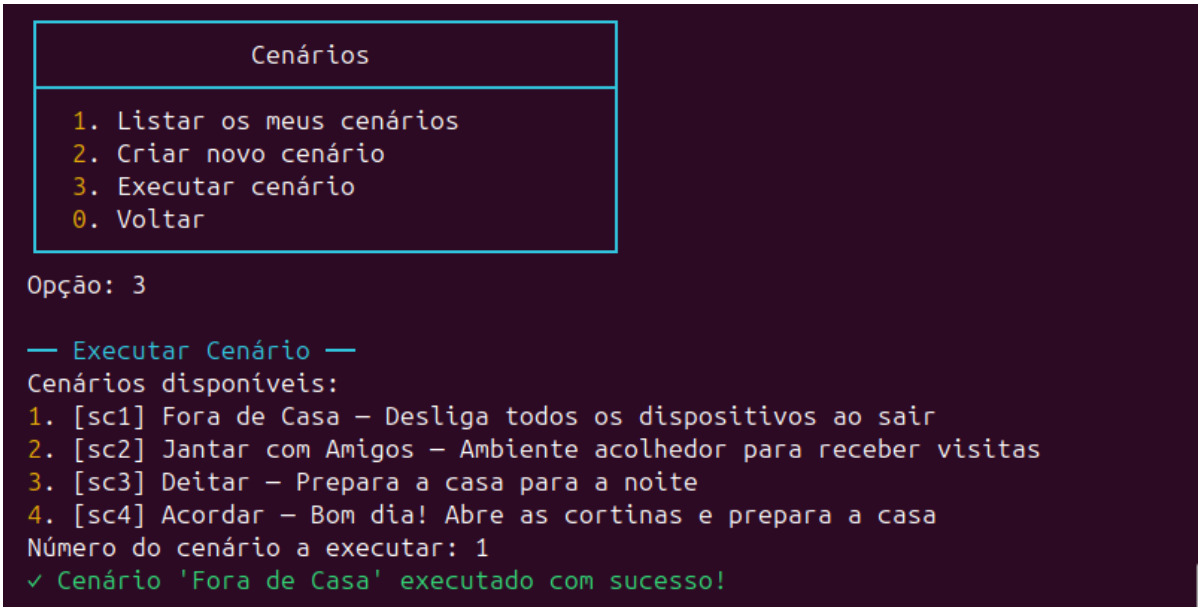


Figura 25 - *Execução do cenário "Fora de Casa" com confirmação de sucesso*

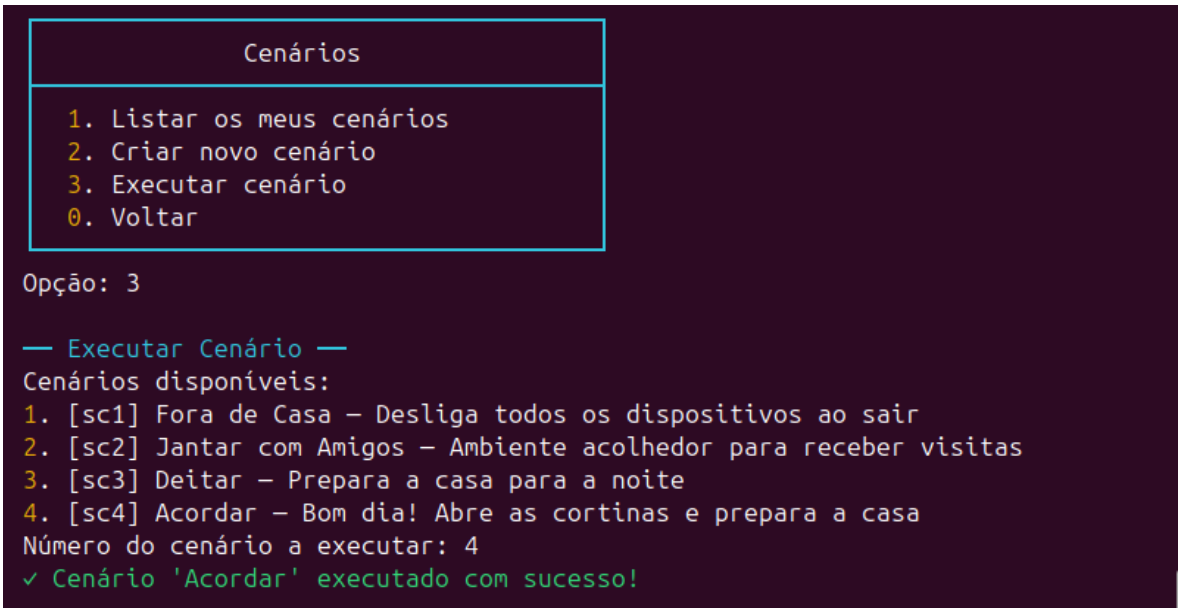


Figura 26 - *Execução do cenário "Acordar" com confirmação de sucesso*

4.7. Estatísticas

As estatísticas são acessíveis no menu principal e incluem: a casa que mais consome energia, o top 3 de dispositivos por número de ativações, o top 3 por tempo ligado, e as 3 divisões com mais dispositivos.

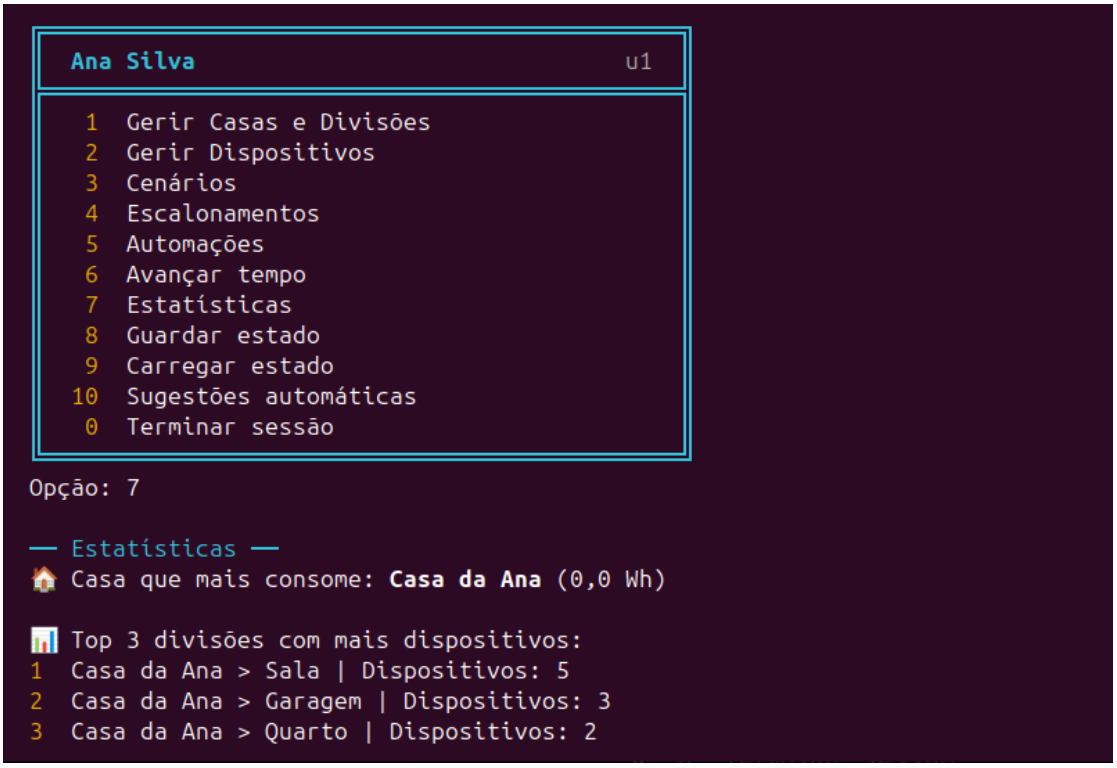


Figura 27 - *Estatísticas de consumo e divisões com mais dispositivos*

```
ID da casa para ver top 3 dispositivos (ou Enter para saltar): c1

Por ativações:
1  Philips HueLamp  |  Ativações: 1
2  Somfy RS100  |  Ativações: 1
3  Philips SceneSwitch  |  Ativações: 1

Por tempo ligado:
1  Philips HueLamp  |  Tempo: 0 min
2  Ikea Tradfri  |  Tempo: 0 min
3  Somfy RS100  |  Tempo: 0 min
```

Figura 28 - *Top 3 dispositivos por número de ativações e por tempo ligado*

4.8. Gravação e Carregamento de Estado

A aplicação permite, em qualquer momento, gravar o estado completo em ficheiro e mais tarde recarregá-lo, retomando a sessão exatamente como foi deixada. Esta funcionalidade está disponível diretamente no menu principal através das opções *"Guardar estado"* e *"Carregar estado"*, sendo o utilizador convidado a indicar o caminho do ficheiro a usar.

Internamente, a operação tira partido do mecanismo de serialização nativa do Java descrito na Secção 2.4. Quando o utilizador grava o estado, o método `gravarEstado(String caminho)` do controlador serializa o próprio `DomusController` para um ficheiro binário. Como o controlador agrega referências para o mapa de utilizadores, o mapa de casas, o relógio simulado e o histórico de interações, e todas estas estruturas contêm transitivamente apenas objetos serializáveis, uma única chamada a `oos.writeObject(this)` é suficiente para preservar toda a informação relevante: as casas com as suas divisões e dispositivos, os estados atuais (ligado/desligado, brilho, abertura, etc.), as métricas acumuladas (`totalMinutosLigado`, `numActivacoes`), as automações e escalonamentos, os cenários de cada utilizador e o histórico que alimenta o motor de sugestões.

O carregamento é feito pelo método estático `carregarEstado(String caminho)`, que devolve uma nova instância de `DomusController` reconstruída a partir do ficheiro. Por ser um método estático, pode ser invocado antes de existir qualquer outro estado em memória, sendo este o ponto de entrada quando o utilizador opta por retomar uma sessão.

Foi também produzido um ficheiro de estado para apresentação, contendo o estado de teste descrito no Capítulo 5, que será carregado durante a defesa do projeto.

4.9. Sugestões Automáticas

Para além das funcionalidades base, a aplicação inclui um mecanismo que sugere automaticamente automações e escalonamentos com base nas interações passadas do utilizador, descrito tecnicamente na Secção 2.1.6. Do ponto de vista do utilizador, esta funcionalidade fica acessível no menu principal através da opção *"Sugestões automáticas"*, que invoca o `SuggestionMenu`.

Ao entrar neste menu, é solicitado o identificador da casa a analisar. O controlador pede então ao `SuggestionEngine` para analisar o histórico de interações dessa casa, e o resultado é apresentado como uma lista numerada onde cada sugestão é identificada com o seu tipo (escalonamento ou automação) e uma descrição em linguagem natural, como por exemplo, *"O dispositivo 'Philips HueLamp' foi ligado 3 vezes às 19h. Criar escalonamento diário às 19:00?"*. Caso o histórico ainda não contenha interações suficientes para detetar padrões, é apresentada uma mensagem informativa a convidar o utilizador a continuar a usar o sistema.

O utilizador escolhe então a sugestão que pretende aceitar (ou **0** para voltar). Quando aceita uma sugestão, o controlador, através do método `aceitarSugestao(idCasa, s)`, adiciona o objeto `Scheduling` ou `Automation` correspondente à casa, integrando-o imediatamente no funcionamento normal do sistema. Passa então a ser verificado nos ciclos de avaliação de automações e escalonamentos como qualquer outra regra criada manualmente. A Figura 29 mostra o menu com sugestões geradas a partir do histórico, e a Figura 30 ilustra a aceitação de uma sugestão.

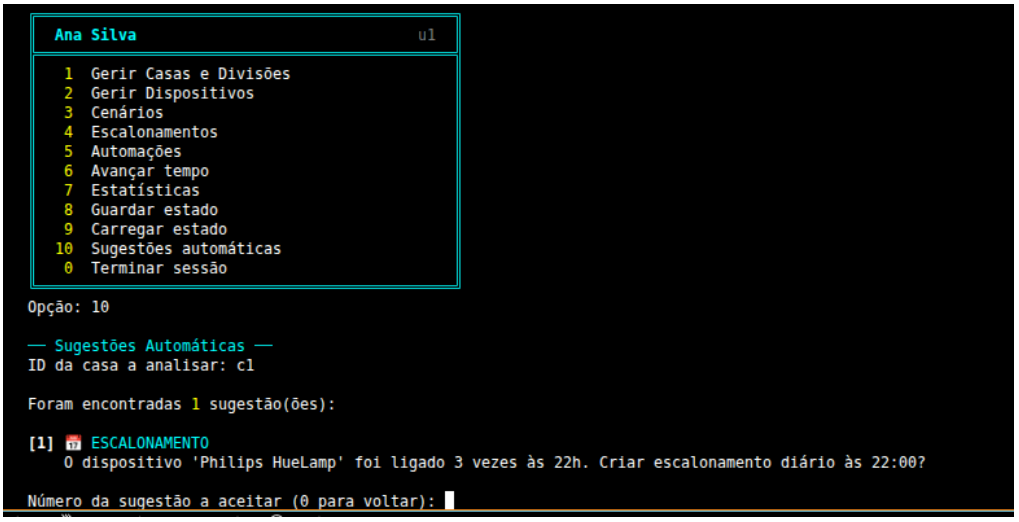


Figura 29 *- Menu de sugestões automáticas com a lista de sugestões geradas*

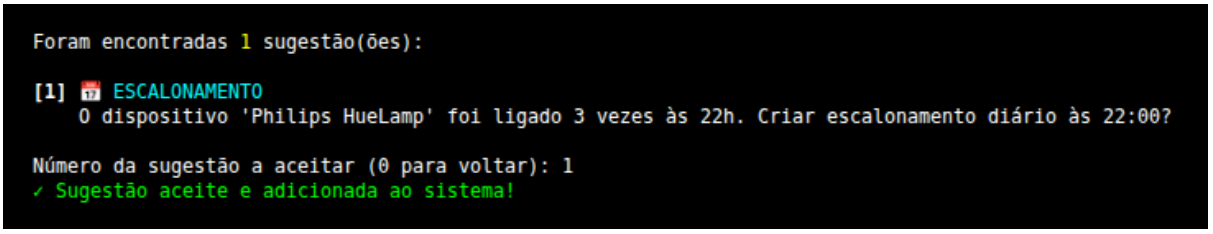


Figura 30 *- Aceitação de uma sugestão e integração no sistema*

5. Estado de Teste

Para garantir uma demonstração eficaz das funcionalidades da aplicação e facilitar os testes durante o desenvolvimento, foi criado um estado de teste com dados suficientes para exercitar todos os patamares de avaliação do projeto. Este estado é gerado pela classe **`TestData`**, do package `utils`, e é carregado automaticamente no arranque da aplicação.

O estado de teste inclui:

- **2 utilizadores** com papéis distintos: Ana Silva (administradora, `u1`) e Bruno Costa (utilizador regular, `u2`);
- **1 casa** , a *"Casa da Ana"*, com 4 divisões (Sala, Quarto, Cozinha e Garagem);
- **11 dispositivos** distribuídos pelas divisões, cobrindo todos os tipos implementados (lâmpada, lâmpada de cor, coluna de som, cortinas, portão de garagem, relé, tomada e sensores de chuva e luminosidade);
- **4 cenários** associados à Ana Silva: *Fora de Casa*, *Jantar com Amigos*, *Deitar* e *Acordar*, conforme exigido pelo enunciado;
- **4 escalonamentos** associados à casa: *Abrir Cortinas de Manhã* (07:30), *Fechar Cortinas à Noite* (21:00), *Ligar Luzes ao Anoitecer* (19:00) e *Desligar Tudo à Meia-Noite* (00:00).

Este conjunto cobre todas as funcionalidades obrigatórias do enunciado e permite demonstrar imediatamente, após o arranque da aplicação, a operação sobre dispositivos, a execução dos cenários, o disparo dos escalonamentos quando o tempo é avançado e o cálculo das estatísticas. A demonstração do motor de sugestões automáticas requer apenas que sejam geradas interações repetidas com o mesmo dispositivo à mesma hora do dia.

6. Conclusão

O *DomusControl* cumpre integralmente os requisitos do enunciado. A arquitetura adotada - Model-View-Controller com encapsulamento rigoroso, polimorfismo aplicado a sete tipos de dispositivo e exceções específicas para cada tipo de erro - permitiu construir o sistema de forma incremental e integrar funcionalidades adicionais com impacto mínimo no código existente.

6.1. Principais Dificuldades

As decisões de design mais delicadas foram a modelação dos utilizadores - a solução intuitiva de criar subclasses revelou-se incompatível com o requisito de papéis diferentes em casas diferentes, levando à adoção do **`Map<String, Role>`** - e a calibração do motor de sugestões, em que definir critérios suficientemente significativos para evitar falsos positivos, mas atingíveis num cenário de demonstração, exigiu testes empíricos até se chegar ao limiar de três ocorrências e à janela de cinco minutos para correlações sensor-atuador.

6.2. Trabalho Futuro

Várias extensões seriam naturais: a substituição da interface em modo texto por uma interface gráfica em JavaFX, beneficiando da separação MVC já existente; o enriquecimento do motor de sugestões com deteção de padrões semanais e correlações entre múltiplos dispositivos; e a introdução de novos tipos de sensor e de automações com múltiplas condições.

6.3. Aprendizagens

O projeto consolidou na prática conceitos que, isoladamente, podiam parecer académicos: o encapsulamento traduzido em construtores de cópia e *getters* defensivos, o polimorfismo que dispensou o uso de `instanceof`, e a separação MVC que tornou a integração de novas funcionalidades quase indolor. O exercício de justificar no relatório não apenas *o que* foi feito, mas *porquê*, revelou-se tão exigente quanto a escrita do código.

ANEXOS

Diagrama UML Completo

